SCALE FOR PROJECT PYTHON MODULE (/PROJECTS/PYTHON-MODULE-07)

You should evaluate 1 student in this team

★

Git repository

```
git@vogsphere-v2.42madrid.com:vogsphere/intra-uuid-b8086a9f-5e1
```

Introduction

- Remain polite, courteous, respectful, and constructive
throughout the evaluation process. The well-being of the community
depends on it.

- Together with the person (or group) being evaluated, identify any issues in the work.
Take the time to discuss and debate the problems you have identified.

- You must consider that there might be differences in how your
peers have understood the project's instructions and the
scope of its functionality. Always keep an open mind and grade
their work as honestly as possible. The pedagogy is valid
only if peer evaluation is conducted seriously.

Guidelines

- Only grade the work that is in the student's or group's
Git repository.

- Double-check that the Git repository belongs to the student
or the group. Ensure that the work is for the relevant project
and also check that the "git clone" command is run in an empty folder.

- Check carefully that no malicious aliases were used to fool you
and make you evaluate something other than the content of the
official repository.

- To avoid any surprises, carefully check that both the evaluating
and the evaluated students have reviewed any scripts that may be used
to facilitate grading.

- If the evaluating student has not completed that particular
project yet, it is mandatory for this student to read the
entire subject document prior to starting the defense.

- Use the flags available on this scale to signal an empty repository,
a non-functioning program, a norm error, cheating, etc. In these cases,
the grading is over and the final grade is 0 (or -42 in the case of
cheating). However, with the exception of cheating, you are
encouraged to continue to discuss your work (even if you have not
finished it) in order to identify any issues that may have caused
this failure and to avoid repeating the same mistakes in the future.

- Remember that for the duration of the defense, no unexpected,
premature, uncontrolled termination of the program is allowed;
otherwise, the final grade is 0. Use the appropriate flag.
You should never have to edit any file other than the configuration file, if one exists.
If you want to edit a file, take the time to discuss the reasons with the
evaluated student and make sure both of you agree with this change.

Attachments

subject.pdf (https://cdn.intra.42.fr/pdf/pdf/203235/en.subject.pdf)

Preliminaries

Basics

- Only grade the work that is in the learner's Git repository
- Ensure the Git repository belongs to the learner
- Check that no malicious aliases are used
- Verify the project structure matches the subject requirements: ex0, ex1, and ex2 exist and co
  file.

Are all preliminary checks satisfied?

Yes No

Exercises

Exercise 0 - Creature Factory

Check the content of the ex0/ folder:

- it contains an **init**.py file and at least one other file, but multiple files are allowed.
- the code defines the Creature abstract class, inheriting from ABC. The attack() method must
  method, and the describe() method must be a concrete method.
- at least four different Creature concrete classes are defined, inheriting from the Creature clas
- the code defines the CreatureFactory abstract class, inheriting from ABC. The create_base()
  create_evolved() abstract methods exist.
- at least two concrete factory classes, one for each family with two Creature each.
- the **init**.py file should only expose the Creature factories, not the Creature themselves.

Note: the subject suggests Creature names, factory names. The evaluated project can use differen
names
as long as the involved design patterns are correctly implemented. This note is valid throughout the

Check that the script "battle.py" is present and follows the established scenario:

- it instantiates both Creature factory classes.
- a single function that accepts any factory exists, and it applies this simple process: create the
  describe it and attack, then repeat with the evolved Creature.
- this function works with both previously created factories.
- another function that receives two factories exists and creates two Creature (base or evolved
  them, and makes them fight.

If everything works correctly, and the abstract factory pattern is correctly implemented, tick "Yes". O
"No".

Yes No

Exercise 1 - Capabilities

Check the content of the ex1/ folder:

- it contains an **init**.py file and at least one other file, but multiple files are allowed.
- the code defines the HealCapability and TransformCapability abstract classes, inheriting fron
  contain the heal() and transform() + revert() abstract methods, respectively. For Creature ent
  capabilities and associated methods are allowed.
- at least four new Creature concrete classes are defined, inheriting from the Creature class pl
  two capability classes above. (two Creature with heal capability, and two Creature with transf
- at least two concrete factory classes, one for each new family with two Creature each.
- the **init**.py file should only expose the Creature factories, not the Creature themselves.

Check that the script "capacitor.py" is present and follows the established scenario:

- it instantiates both new Creature factory classes.
- each factory is tested: the standard methods describe() and attack() are used, along with the
  capability inserted at the correct place in the process (see the subject for the suggested Crea
  capabilities). This applies to both the base and the evolved Creature.

If everything works correctly, and the abstract capabilities are correctly implemented, tick "Yes". Oth
"No".

Yes No

Exercise 2 - Abstract Strategy

Check the content of the ex2/ folder:

- it contains an **init**.py file and at least one other file, but multiple files are allowed.
- the code defines the BattleStrategy abstract class, inheriting from ABC. The act() and is_valid abstract methods.
- at least three different concrete strategy classes are defined, inheriting from the BattleStrateg
- a "normal" strategy exists and is suitable for any Creature, whatever the family or the capabil
- different strategies exist and match the capabilities from ex1. For example, a defensive strate with a Creature with HealCapability and then call heal() after the attack().

Check that the script "tournament.py" is present and follows the established scenario:

- it instantiates Creature factory classes from ex0 and ex1.
- it instantiates the different strategies available.
- a single battle() function exists and handles the tournament (you can create sub-functions).
- the battle() function accepts, as a parameter, a list of tuples consisting of a CreatureFactory a BattleStrategy.
- the battle() function organizes a fight between each pair of opponents from the list. Check tha does not fight itself, and does not fight another Creature twice (A-B and B-A).
- Verify that its associated strategy is applied to the Creature during the fight. For example, a h will heal() when using a defensive strategy, but not when using a normal strategy.
- Check that a unsuitable strategy for a Creature stops the tournament.

If everything works correctly, and the abstract strategy pattern is correctly implemented, tick "Yes". ("No".

Yes No

Code Quality and Understanding

Code Quality and Understanding

Assess the overall code quality and standards:

- Is the code written in Python 3.10 or higher?
- Does the code adhere to the flake8 linter standards (no errors)?
- Type hints are REQUIRED for all functions and methods; check them using mypy.
- Docstrings are NOT required for this module.
- Is the code well-structured and readable?

Understanding:

- Does the code demonstrate proper use and understanding of abstract patterns?

Answer "Yes" if all quality standards are met, and "No" if any are lacking.

Yes No

Ratings

Don’t forget to check the flag corresponding to the defense

✔ Ok ★ Outstanding project

Empty work Incomplete work Invalid compilation Norme Cheat

Concerning situation Leaks Forbidden function Can’t support /

Conclusion

Leave a comment on this evaluation (2048 chars max)

Finish evaluation

API General Terms of Use (https://profile.intra.42.fr/legal/terms/33)
Declaration on the use of cookies (https://profile.intra.42.fr/legal/terms/2)
Privacy policy (https://profile.intra.42.fr/legal/terms/5)
General term of use of the site (https://profile.intra.42.fr/legal/terms/6)
Rules of pro (https://profile.intra.42